• COP 4710 · DATABASE SYSTEMS • MIDTERM 2 · STUDY GUIDE

Midterm 2 — Study Guide & Solved Examples

Everything on the two-page formula sheet, organized by topic with the exam's official problems worked out: DML & DDL, joins and aggregates, PL/pgSQL functions and exceptions, arrays/strings/dates, views, indexes & B+ trees, and transactions/locking/permissions. PostgreSQL / PL-pgSQL.

Working schema used throughout. Two tables, self-referencing manager and department head:

```
Departments(dept_id PK, dept_name, dept_head -> Employees.emp_id,
            budget, city)
Employees(emp_id PK, fname, lname, salary,
          dept_id -> Departments.dept_id,
          manager_id -> Employees.emp_id)
```

ORIENTATION

01 Pick the Right Tool

Most questions are 'make this change' or 'answer this question about the data.' Match the ask to the construct, then follow the recipe.

IF THE QUESTION ASKS YOU TO…	USE
Change a table's structure (a column, a constraint)	`ALTER TABLE` (DDL)
Change values in existing rows	`UPDATE` (DML)
Remove rows	`DELETE` (DML)
Choose rows via another table — matched / none	`IN` / `EXISTS` · `NOT IN` / `NOT EXISTS`
Filter rows vs. filter groups	`WHERE` (rows) vs. `HAVING` (groups)
Keep unmatched left-side rows	`LEFT JOIN`
Hide columns / save a reusable query	`VIEW`
Validate input and return status codes	function + `EXCEPTION` block
Several changes, all-or-nothing	transaction or a function
Auto-react to a change	trigger
Faster lookup	index (hash = equality, B+ tree = range)

Recipe — change/delete rows chosen through another table

- Write the inner `SELECT key FROM other WHERE …` and mentally run it alone.
- Outer: `WHERE key IN (inner)` for rows that **have** a match, or `NOT EXISTS (… WHERE o.key = t.key)` for rows with **none**.
- With `EXISTS`, don't forget the keyword **and** the correlation line (`d.dept_id = e.dept_id`).
- `SET` column takes **no alias**; `UPDATE` takes **no column list**; `DELETE` takes **no JOIN** (use `USING` or a subquery).
- Re-read: is there a `WHERE`? Which rows survive on the sample data?

Recipe — write a function

- Signature, `RETURNS` type, dollar quotes, `LANGUAGE plpgsql`.
- `DECLARE` variables (`:=` to assign; every statement ends `;`).
- Check in order: **arguments** (NULL?) → **existence** (`SELECT … INTO` / `EXISTS`) → **value**.

- Put a risky cast inside its own `BEGIN … EXCEPTION … END;`.
- Do the work, then `RETURN`. **Every path returns.**

INSERT · UPDATE · DELETE

02 Data Manipulation (DML)

Add, change, and remove rows. The exam's favorite move: pick the affected rows through a subquery on the other table.

Syntax forms

```
INSERT INTO t (c1,c2) VALUES (v1,v2), (v3,v4);
INSERT INTO t (c1,c2) SELECT a,b FROM s WHERE ..;  -- list count must match
UPDATE t SET c1 = v1, c2 = v2 WHERE cond;
DELETE FROM t WHERE cond;
UPDATE t SET c = s.c FROM s WHERE t.k = s.k;        -- update using another table
DELETE FROM t USING s WHERE t.k = s.k;              -- delete using another table
```

- `UPDATE`: `WHERE` picks rows first; the `SET` right side uses the **old** values. `SET` column has no alias (`SET salary`, not `SET e.salary`); there is **no column list** after the table name.
- Omitted `INSERT` columns get their `DEFAULT`, else `NULL`.
- `DELETE` allows no `JOIN` — use `USING` or a subquery. Don't repeat the target table in `UPDATE … FROM`.
- **Traps:** no `WHERE` = every row. `NOT IN` with a `NULL` in the subquery returns nothing → use `NOT EXISTS`. `= NULL` is never true → use `IS NULL`. A scalar subquery must return exactly 1 row.
- `ILIKE` = case-insensitive `LIKE`. `%` = any run (even empty), `_` = one char; no `%` means the whole string must match.

OFFICIAL Q1 **UPDATE + ILIKE + subquery**

For employees of departments whose name contains “Research” (case-insensitive), double the salary.

```
UPDATE Employees
SET salary = salary * 2
WHERE dept_id IN (
    SELECT dept_id
    FROM Departments
    WHERE dept_name ILIKE '%research%'
);
```

▸ Inner query lists the matching department ids; the outer `UPDATE` hits employees in those departments.

Q1 VARIANT

Same idea with EXISTS

Double salaries for employees whose department name contains “Sales”.

```
UPDATE Employees e
SET salary = salary * 2
WHERE EXISTS (
    SELECT 1
    FROM Departments d
    WHERE d.dept_id = e.dept_id      -- correlation line
      AND d.dept_name ILIKE '%sales%'
);
```

▶ `EXISTS` needs the correlation line tying `d.dept_id` to `e.dept_id`; otherwise it's true for every row.

OFFICIAL Q2

DELETE + NOT EXISTS

Delete departments with budget 0 AND no employees.

```
DELETE FROM Departments d
WHERE d.budget = 0.00
  AND NOT EXISTS (
      SELECT 1
      FROM Employees e
      WHERE e.dept_id = d.dept_id
  );
```

▶ “No children” = `NOT EXISTS`. A scalar-count variant works too: `AND (SELECT COUNT(*) FROM Employees e WHERE e.dept_id = d.dept_id) = 0`.

VARIANT

UPDATE from an aggregate condition

Raise salaries 5% in departments whose average salary is under 5000.

```
UPDATE Employees
SET salary = salary * 1.05
WHERE dept_id IN (
    SELECT dept_id
    FROM Employees
    GROUP BY dept_id
    HAVING AVG(salary) < 5000.00
);
```

▶ Group/aggregate lives **inside** the subquery; the outer `UPDATE` just filters by the ids it returns.

VARIANT

Correlated / scalar subquery in SET

Set each department head's salary to that department's maximum salary.

```
UPDATE Employees e
SET salary = (
    SELECT MAX(e2.salary)
    FROM Employees e2
    WHERE e2.dept_id = e.dept_id
)
WHERE e.emp_id IN (
    SELECT dept_head FROM Departments
    WHERE dept_head IS NOT NULL
);
```

▶ The `SET` value is a scalar subquery (one row). Guard the rows with `WHERE` so only heads are touched.

DDL · STRUCTURE

03 Changing Structure (ALTER TABLE)

DDL that changes the table itself; existing rows are kept. Column adds, drops, renames, type/default/not-null changes, and constraints.

```
ALTER TABLE t ADD COLUMN c TYPE [NOT NULL] [DEFAULT v];
ALTER TABLE t DROP COLUMN c [CASCADE];
ALTER TABLE t RENAME COLUMN old TO new;   -- or RENAME TO newtable
ALTER TABLE t ALTER COLUMN c TYPE T [USING c::T];
ALTER TABLE t ALTER COLUMN c SET DEFAULT v;   -- or DROP DEFAULT
ALTER TABLE t ALTER COLUMN c SET NOT NULL;    -- or DROP NOT NULL
ALTER TABLE t ADD CONSTRAINT n CHECK (..) | UNIQUE (c)
      | PRIMARY KEY (c) | FOREIGN KEY (c) REFERENCES p (k) [ON DELETE act];
ALTER TABLE t DROP CONSTRAINT n;
```

- Several actions can be comma-separated in one `ALTER`. `RENAME` can't be combined with others.
- `NOT NULL` and `DEFAULT` use `ALTER COLUMN`, not `ADD CONSTRAINT`.
- **ALTER fails when:** `SET NOT NULL` with NULLs present (fix the rows first); `ADD CONSTRAINT` with rule-breaking rows; a `TYPE` value won't convert/fit (use `USING`); `ADD COLUMN NOT NULL` on a filled table with no `DEFAULT`; `DROP COLUMN` blocked by a dependent view (`CASCADE` drops it).
- `SET DEFAULT` affects future rows only. `DROP COLUMN` also drops that column's own constraints; an FK elsewhere pointing at it blocks the drop.

OFFICIAL Q3

Add a NOT NULL constraint

Add NOT NULL to Departments.dept_name.

```
ALTER TABLE Departments
ALTER COLUMN dept_name SET NOT NULL;
```

▶ If any row already has a NULL name, `UPDATE … WHERE dept_name IS NULL` first, or this fails.

OFFICIAL Q4

Drop a column

Remove manager_id from Employees.

```
ALTER TABLE Employees DROP COLUMN manager_id;
```

▶ Any view or FK referencing `manager_id` must be dropped first (or use `CASCADE`).

OFFICIAL Q5

Add a column with default

Add salary_expectation DECIMAL(15,2), default 0.00, not null.

```
ALTER TABLE Employees
ADD COLUMN salary_expectation DECIMAL(15,2)
    NOT NULL DEFAULT 0.00;

-- if NULLs must be allowed, drop NOT NULL:
ALTER TABLE Employees
ADD COLUMN salary_expectation DECIMAL(15,2) DEFAULT 0.00;
```

▶ Adding a `NOT NULL` column to a filled table **requires** a `DEFAULT` so existing rows have a value.

VARIANTS

Change / drop default, add a CHECK

```
ALTER TABLE Employees ALTER COLUMN salary_expectation SET DEFAULT 1000.00;
ALTER TABLE Employees ALTER COLUMN salary_expectation DROP DEFAULT;
ALTER TABLE Employees
ADD CONSTRAINT chk_salary_nonnegative CHECK (salary >= 0.00);
```

▶ To change an existing constraint you **DROP** it then **ADD** it again — there is no “ALTER CONSTRAINT.”

REFERENTIAL ACTIONS

04 Foreign-Key Actions

What happens to child rows when a referenced parent row is deleted or updated.

OPTION	EFFECT ON CHILD ROWS
`NO ACTION` (default)	The delete/update of the parent is **rejected** if children exist.
`CASCADE`	Children are deleted too / get the new key.
`SET NULL`	Key column set to `NULL` (not allowed if it's part of the child's PK).
`SET DEFAULT`	Key column set to its declared default.

- Change an action by `DROP`ping the constraint and `ADD`ing it again. Default name is `table_column_fkey`.
- A `manager_id` FK with `NO ACTION` blocks deleting someone who still has reports.
- **Circular FKs** (`employees.dept_id ↔ departments.dept_head`): create one table with a nullable column and add its FK later with `ALTER`, or use `DEFERRABLE` + `SET CONSTRAINTS … DEFERRED`.

JOIN · GROUP BY · HAVING

05 Joins, GROUP BY, HAVING

Combine tables and summarize groups. The classic exam query: per-department counts and salary stats with a HAVING filter.

- Clause order: `SELECT` · `FROM` · `WHERE` (rows) · `GROUP BY` · `HAVING` (groups, aggregates OK) · `ORDER BY`. `WHERE` cannot use aggregates.
- `INNER JOIN` keeps matches only. `LEFT JOIN` keeps all left rows; right columns are `NULL` when there's no match. Self-join = same table, two aliases (`e.manager_id = m.emp_id`).
- Every non-aggregate `SELECT` column must appear in `GROUP BY`. Qualify shared names (`d.dept_id`).
- On a LEFT JOIN with no match: `COUNT(*)` still counts the padded row (1); `COUNT(right.col)` gives 0. A `HAVING` on `AVG/SUM` drops empty LEFT groups (`NULL > 5000` isn't true) — use `COALESCE` if you need them.

OFFICIAL Q10 **Per-department stats with HAVING**

department_name, employees_no, max_salary — only departments whose average salary > 5000.

```
SELECT d.dept_name AS department_name,
       COUNT(e.emp_id) AS employees_no,
       MAX(e.salary)   AS max_salary
FROM Departments d
JOIN Employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 5000.00;
```

▸ Filter on an aggregate (`AVG`) belongs in `HAVING`, not `WHERE`. Group by the PK plus any selected label column.

VARIANT **Include zero-employee departments**

Every department must appear, showing 0 employees / 0 payroll.

```
SELECT d.dept_name,
       COUNT(e.emp_id) AS employees_no,
       COALESCE(SUM(e.salary), 0.00) AS total_salary
FROM Departments d
LEFT JOIN Employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```

▸ `LEFT JOIN` keeps empty departments; `COALESCE` turns the `NULL` sum into 0.

VARIANT

Above own-department average

Employees earning more than their department's average.

```
SELECT e.emp_id, e.fname, e.lname, e.salary
FROM Employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employees e2
    WHERE e2.dept_id = e.dept_id
);
```

▶ Correlated subquery recomputes the average for each employee's own department.

VARIANTS

More HAVING patterns

```
-- payroll exceeds budget
SELECT d.dept_name, d.budget, SUM(e.salary) AS payroll
FROM Departments d JOIN Employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name, d.budget
HAVING SUM(e.salary) > d.budget;

-- max salary at least twice the min
SELECT dept_id
FROM Employees
GROUP BY dept_id
HAVING MAX(salary) >= 2 * MIN(salary);
```

▶ Anything comparing group aggregates goes in `HAVING`; include grouped columns you also select.

DO · FUNCTIONS · LOOPS

06 PL/pgSQL: Blocks, Loops, Functions

Procedural code: a DO block runs once; a FUNCTION is a reusable named block that returns a value.

DO block, loop, IF

```
DO $b$
DECLARE _total INTEGER := 0;
BEGIN
  FOR k IN 1..10 LOOP _total := _total + k; END LOOP;
  IF _total > 50 THEN RAISE INFO 'big %', _total;
  ELSIF _total = 0 THEN RAISE INFO 'none';
  ELSE RAISE INFO 'small %', _total;
  END IF;
END;
$b$ LANGUAGE plpgsql;
```

- Dollar-quote tags must match (`$b$ … $b$`). `:=` assigns; every statement ends `;`.
- `RAISE INFO '..%..', v;` — each `%` consumes the next argument.
- `EXIT` leaves the loop; `RETURN` ends the whole block/function. It's `ELSIF` (not ELSEIF); `IF` needs `THEN … END IF;`. The `FOR` loop variable is its own variable; a NULL loop bound is an error.

Function shape

```
CREATE OR REPLACE FUNCTION public.f_Name(_p TYPE, _o TYPE = dflt)
RETURNS TYPE AS
$fb$
DECLARE _v TYPE := init;
BEGIN
  ...
  RETURN value;        -- every path returns
END;
$fb$ LANGUAGE plpgsql;

DROP FUNCTION IF EXISTS public.f_Name(VARCHAR(20));  -- identify by arg types
SELECT dept_name, f_Name(dept_name) FROM Departments;
```

- Defaults go at the **end** of the parameter list. Prefix variable names with `_` to avoid clashing with column names.
- Look up with `SELECT c INTO _v FROM t WHERE ..;` — leaves `_v` NULL if no row. `MAX`/`MIN` over 0 rows = NULL; `COUNT` over 0 rows = 0.
- Identify a function to `DROP` by its argument types.

REVIEW Q12

Count near-top earners (95% of max)

Return how many employees in a department earn at least 95% of that department's max salary.

```
CREATE OR REPLACE FUNCTION f_getLargestSalariedEmployees(_dept_id SMALLINT)
RETURNS SMALLINT AS
$FBODY$
DECLARE
  _max_salary DECIMAL(10,2) := NULL;
  _emp_no     INTEGER := NULL;
BEGIN
  IF _dept_id IS NULL THEN RETURN -1; END IF;          -- bad argument

  SELECT MAX(e.salary) INTO _max_salary
  FROM Employees e WHERE e.dept_id = _dept_id;

  IF _max_salary IS NULL THEN RETURN 0; END IF;        -- no employees

  SELECT COUNT(emp_id) INTO _emp_no
  FROM Employees
  WHERE dept_id = _dept_id
    AND salary >= 0.95 * _max_salary;

  RETURN _emp_no;
END;
$FBODY$ LANGUAGE plpgsql PARALLEL SAFE;
```

▶ Check order: argument → existence/data (max is NULL ⇒ empty) → compute. Sentinel `-1` for bad input, `0` for an empty department.

VARIANT

CASE on a count / salary deviation

```
SELECT COUNT(*) INTO _cnt FROM Employees WHERE dept_id = _dept_id;
CASE
  WHEN _cnt = 0 THEN RETURN 'Empty';
  WHEN _cnt < 5 THEN RETURN 'Small';
  ELSE RETURN 'Large';
END CASE;

-- MAX - MIN spread, sentinel if empty
SELECT MAX(salary) - MIN(salary) INTO _d FROM Employees WHERE dept_id = _dept_id;
IF _d IS NULL THEN RETURN -1.00; END IF;
RETURN _d;
```

▶ `CASE … END CASE;` for multi-way branching; the aggregate-is-NULL test detects an empty group.

VALIDATION · SENTINELS

07 Exceptions & Input Validation

Validate before you act, wrap risky casts, and return negative codes instead of crashing.

- Check order: **arguments** (NULL = bad argument) → **existence** → **value** (negative/FALSE = bad value).
- Sentinels are negative codes (impossible real answers, since a real salary is positive). State each code's meaning in a comment.
- A failed cast aborts the statement — wrap it in its own `BEGIN … EXCEPTION … END;`. `'Hello'::INTEGER` raises `invalid_text_representation`; `5000000000::INTEGER` (or a value past `NUMERIC(10,2)`'s max 99999999.99) raises `numeric_value_out_of_range`.
- After an `UPDATE`, `IF NOT FOUND THEN …` catches "no such row."

OFFICIAL Q6

f_SetSalary — validate string, set, return

f_SetSalary(_emp_id BIGINT, _new_salary VARCHAR(20)): validate id and salary text; on success return the salary, on errors return negative codes.

```
CREATE OR REPLACE FUNCTION f_SetSalary(_emp_id BIGINT, _new_salary VARCHAR(20))
RETURNS DECIMAL(10,2) AS
$FBODY$
DECLARE _salary DECIMAL(10,2) := NULL;
BEGIN
  IF _emp_id IS NULL
     OR NOT EXISTS (SELECT 1 FROM Employees WHERE emp_id = _emp_id)
  THEN RETURN -1.00; END IF;              -- bad id / no such employee

  IF _new_salary IS NULL THEN RETURN -2.00; END IF;

  BEGIN                                   -- risky cast in its own block
    _salary := TRIM(_new_salary)::DECIMAL(10,2);
  EXCEPTION
    WHEN invalid_text_representation THEN RETURN -2.00;  -- '12abc', ''
    WHEN numeric_value_out_of_range  THEN RETURN -3.00;  -- too big
  END;

  IF _salary < 0 THEN RETURN -4.00; END IF;               -- bad value

  UPDATE Employees SET salary = _salary WHERE emp_id = _emp_id;
  RETURN _salary;
END;
$FBODY$ LANGUAGE plpgsql;
```

▸ The cast lives in a nested block so a bad string returns a code instead of aborting. Codes are negative because a real salary is positive.

VARIANT

UPDATE … RETURNING to confirm the write

```
CREATE OR REPLACE FUNCTION f_SetSalary2(_emp_id INTEGER, _new VARCHAR(20))
RETURNS DECIMAL(10,2) AS $$
DECLARE _salary DECIMAL(10,2);
BEGIN
  BEGIN
    _salary := _new::DECIMAL(10,2);
  EXCEPTION WHEN OTHERS THEN RETURN -1.00;
  END;

  UPDATE Employees SET salary = _salary
  WHERE emp_id = _emp_id
  RETURNING salary INTO _salary;          -- value actually stored

  IF NOT FOUND THEN RETURN -2.00; END IF;  -- no such employee
  RETURN _salary;
END;
$$ LANGUAGE plpgsql;
```

▶ `RETURNING … INTO` reads back what PostgreSQL stored; `NOT FOUND` flags a missing employee.

TRACE

String → integer status (check_number pattern)

```
'3781957'  ->  positive integer
'0'        ->  integer, but not positive
'007'      ->  positive (cast gives 7)
'12a4'     ->  NOT positive; trouble at position 3
'-5'       ->  NOT positive; '-' is ASCII 45, at position 1
''         ->  NOT positive; empty string or NULL
```

▶ Loop characters; any outside ASCII 48–57 (`'0'`–`'9'`) sets a flag, records the position, and `EXIT`s. Digits '0'–'9' = 48–57.

ARRAY · EXTRACT · STRING FNS

08 Arrays, Dates & Strings

1-based arrays, date-part extraction, and the string functions you'll need for validation and labels.

FUNCTION	DOES
`a || b`	concatenate text
`LOWER / UPPER / LENGTH`	case / length
`TRIM / LTRIM / RTRIM`	strip both / leading / trailing
`POSITION('x' IN s)`	1-based index, 0 = not found
`LEFT(s,n) / RIGHT(s,n)`	first / last n chars
`SUBSTRING(s FROM 4 FOR 8)`	slice from position 4, length 8
`ASCII('R')=82 · CHR(82)='R'`	char ↔ code; digits '0'–'9' = 48–57

- Arrays are **1-based**; `CARDINALITY(arr)` is the length (NULL array → NULL). Test membership with `x = ANY(arr)`.
- `EXTRACT(MONTH FROM d)`, `EXTRACT(DAY FROM d)`, `EXTRACT(DOY FROM d)` pull date parts. Fixed month/day only — moving holidays (Thanksgiving) can't use a fixed date.

OFFICIAL Q11

f_IsHoliday — fixed holidays via arrays

Return TRUE if the date is a fixed holiday.

```
CREATE OR REPLACE FUNCTION f_IsHoliday(_date DATE)
RETURNS BOOLEAN AS
$FBODY$
DECLARE
  _h INTEGER[] := ARRAY[101, 704, 1111, 1225];  -- month*100 + day
  _k INTEGER;
BEGIN
  IF _date IS NULL THEN RETURN FALSE; END IF;
  _k := EXTRACT(MONTH FROM _date) * 100 + EXTRACT(DAY FROM _date);
  FOR _i IN 1..CARDINALITY(_h) LOOP
    IF _h[_i] = _k THEN RETURN TRUE; END IF;
  END LOOP;
  RETURN FALSE;                                  -- nothing matched
END;
$FBODY$ LANGUAGE plpgsql;
```

▶ Encode each holiday as `month*100+day` so one integer compare per entry. Short form: `RETURN _k = ANY(_h);`

VARIANT

One text array with TO_CHAR

```
CREATE OR REPLACE FUNCTION f_IsHoliday2(_date DATE)
RETURNS BOOLEAN AS $$
DECLARE _holidays TEXT[] := ARRAY['01-01','07-04','12-25'];
BEGIN
  IF _date IS NULL THEN RETURN FALSE; END IF;
  RETURN TO_CHAR(_date,'MM-DD') = ANY(_holidays);
END;
$$ LANGUAGE plpgsql;
```

▶ `TO_CHAR(d,'MM-DD')` formats the date; `= ANY` checks the array in one line.

STORED QUERIES · SECURITY

09 Views

A view is a stored query whose rows are computed on use. Great for hiding columns and saving a join.

```
CREATE VIEW v (c1,c2) AS SELECT ...;    DROP VIEW v;
```

- Rows are computed each time the view is used; dropping it leaves the base tables untouched. `LEFT JOIN`s inside keep rows like the top manager (no manager) or employees with no department.
- Uses: logical data independence, security, convenience.
- **Updatable** (SQL-92): one base table, selection + projection only, no aggregates. An insert puts `NULL` in hidden columns (fails if a hidden column is PK/NOT NULL). `WITH CHECK OPTION` rejects changes that would leave the view's condition.
- A view that uses a column blocks `DROP COLUMN` of it until the view is dropped (or `CASCADE`).

OFFICIAL Q8 **Employee info view (two LEFT JOINs)**

View: employee id, full name, department name, salary, manager's full name.

```
DROP VIEW IF EXISTS Employee_Info;
CREATE OR REPLACE VIEW Employee_Info AS
SELECT e.emp_id,
       e.fname || ' ' || e.lname AS full_name,
       d.dept_name,
       e.salary,
       COALESCE(m.fname || ' ' || m.lname, 'N/A') AS manager_full_name
FROM Employees e
JOIN Departments d ON d.dept_id = e.dept_id
LEFT JOIN Employees m ON m.emp_id = e.manager_id;   -- self-join for manager
```

▶ Self-join `Employees m` gets the manager. `LEFT JOIN` keeps the top manager (no manager row); `INNER` would drop them.

QUIZ 2 PATTERN

f_GetDeptInfo — full department info string

Return a sentence describing a department: name, head, employee count, average salary; handle NULL id, missing department, and empty department.

```
CREATE OR REPLACE FUNCTION f_GetDeptInfo(p_dept_id INT)
RETURNS TEXT AS $$
DECLARE
  v_dept_name VARCHAR(100);  v_dept_head INT;
  v_head_name VARCHAR(200);  v_emp_count INT;
  v_avg_salary DECIMAL(15,2); v_exists INT;
BEGIN
  IF p_dept_id IS NULL THEN RETURN 'Department ID cannot be NULL.'; END IF;

  SELECT COUNT(*) INTO v_exists FROM Departments WHERE dept_id = p_dept_id;
  IF v_exists = 0 THEN
    RETURN 'Department ' || p_dept_id || ' does not exist';
  END IF;

  SELECT dept_name, dept_head INTO v_dept_name, v_dept_head
  FROM Departments WHERE dept_id = p_dept_id;

  IF v_dept_head IS NULL THEN
    v_head_name := 'N/A';
  ELSE
    SELECT COALESCE(fname || ' ' || lname, 'N/A') INTO v_head_name
    FROM Employees WHERE emp_id = v_dept_head;
    IF NOT FOUND OR v_head_name IS NULL THEN v_head_name := 'N/A'; END IF;
  END IF;

  SELECT COUNT(*), COALESCE(AVG(salary), 0.00)
  INTO v_emp_count, v_avg_salary
  FROM Employees WHERE dept_id = p_dept_id;

  IF v_emp_count = 0 THEN
    RETURN 'The department ' || v_dept_name ||
           ' managed by ' || v_head_name || ' has 0 employees';
  ELSE
    RETURN 'The department ' || v_dept_name ||
           ' managed by ' || v_head_name ||
           ' has ' || v_emp_count ||
           ' employees; avg salary ' || v_avg_salary;
  END IF;
END;
$$ LANGUAGE plpgsql;
```

▸ Check order again: NULL argument → existence → data. `COALESCE` protects the head name; the empty-department branch is separate from the “doesn't exist” branch.

VARIANT **Gather it all in one grouped query**

```
SELECT d.dept_name,
       COALESCE(h.fname || ' ' || h.lname, 'N/A'),
       COUNT(e.emp_id),
       COALESCE(AVG(e.salary), 0.00)
INTO v_name, v_head, v_count, v_avg
FROM Departments d
LEFT JOIN Employees h ON h.emp_id = d.dept_head    -- the head
LEFT JOIN Employees e ON e.dept_id = d.dept_id      -- members
WHERE d.dept_id = p_dept_id
GROUP BY d.dept_id, d.dept_name, h.fname, h.lname;
IF NOT FOUND THEN RETURN 'Department does not exist'; END IF;
```

▸ Two LEFT JOINs (head, members) collect every field in one `SELECT … INTO`; group by the department plus the head's name columns.

REVIEW Q13 **Department view with head + totals**

Named-column view: dept name, employee count, head last/first, budget, total salary; exclude Hamburg and Barcelona.

```
DROP VIEW IF EXISTS Department_Info;
CREATE OR REPLACE VIEW Department_Info
  (dept_name, employee_no, dept_head_lname, dept_head_fname, budget, total_salary)
AS
SELECT d.dept_name, COUNT(e.emp_id), h.lname, h.fname, d.budget, SUM(e.salary)
FROM Departments d
INNER JOIN Employees e ON e.dept_id = d.dept_id
LEFT JOIN Employees h ON h.emp_id = d.dept_head
WHERE d.city NOT IN ('Hamburg', 'Barcelona')
GROUP BY d.dept_name, h.lname, h.fname, d.budget;
```

▸ You can name the view's columns in parentheses. `INNER JOIN` to members drops empty departments; swap to `LEFT JOIN` + `COALESCE` to keep them.

KEYS · B+ TREES

10 Indexes, Keys & B+ Trees

Keys are constraints; indexes are performance structures. Know which index answers which query.

- **Superkey** ⊇ a key; **candidate key** is minimal; **primary** is the chosen one. Keys = constraints; an index = a performance structure, and its search key need not be unique.
- **Clustered**: data stored in index order — at most one per table, so range scans read a few contiguous pages. **Secondary**/unclustered: other indexes; a range match costs ~1 page read per match (can lose to a full scan). **Index-only**: answered from index entries alone (works even unclustered).
- **Hash**: equality on the **full** key only. **B+ tree**: equality and range, on a **key prefix** — a composite `<age,sal>` can't help `sal > 40` (sal isn't a prefix).
- B+ tree search cost = root-to-leaf path + leaf pages holding matches (leaves are linked). Balanced, height ~3–4. Composite index is bigger and must be updated whenever any of its fields changes. The SQL standard has no `CREATE INDEX` (it's DBMS-specific).
- Index data-entry forms: (1) the record itself, (2) `<key, rid>`, (3) `<key, rid-list>`.

WORKED **B+ tree search & range**

```
Root:   13 | 17 | 24 | 30
Leaves (linked):
 [2* 3* 5* 7*] -> [14* 16*] -> [19* 20* 22*]
 -> [24* 27* 29*] -> [33* 34* 38* 39*]
 ranges:  <13 | 13<=k<17 | 17<=k<24 | 24<=k<30 | k>=30

Search 15 : 13 <= 15 < 17  -> 2nd leaf [14 16] -> not there = absent.
Range 17..27 : find leaf for 17, then follow 'next' links.
Cost = path length (root->leaf) + number of leaves read.
Equality on full key -> hash. Range or key prefix -> B+ tree.
```

▶ Navigate root to the correct leaf by range, then walk linked leaves for a range query.

ACID · LOCKING · TRIGGERS

11 Transactions, Locking & Triggers

All-or-nothing units of work, the anomalies isolation levels prevent, and how to do two updates atomically.

PROPERTY	MEANING
Atomicity	all or nothing
Consistency	consistent in → consistent out
Isolation	result equals some serial order
Durability	committed work survives a crash

Anomalies vs. isolation level

ISOLATION LEVEL	DIRTY READ	UNREPEATABLE READ	PHANTOM
READ UNCOMMITTED	maybe	maybe	maybe
READ COMMITTED	no	maybe	maybe
REPEATABLE READ	no	no	maybe
SERIALIZABLE	no	no	no

- Conflicts (same object, at least one write): **WR** = dirty read (read uncommitted write), **RW** = unrepeatable read, **WW** = overwrite uncommitted data. A **phantom** is a new matching row appearing between two reads.
- **Strict 2PL**: shared (S) lock to read, exclusive (X) lock to write; hold all locks until commit/abort. A deadlock is a wait cycle — the DBMS aborts one transaction.
- `COMMIT`, `ROLLBACK`, `SAVEPOINT s`, `ROLLBACK TO SAVEPOINT s` (undoes work after `s`). `BEGIN;` opens a transaction explicitly; a single function call is itself one atomic unit.
- **Trigger** = Event + Condition + Action; `before`/`after`/`instead of`; per row or per statement. Stored-procedure parameters: `IN`, `OUT`, `INOUT`.

OFFICIAL Q9

Move a department head atomically

Set Departments.dept_head and the head employee's dept_id as one unit; abort on any failure.

```
DO $BODY$
DECLARE
  _head_id INTEGER := 2317;
  _dept_id SMALLINT := 19;
  _returned INTEGER;
BEGIN
  UPDATE Departments SET dept_head = _head_id
  WHERE dept_id = _dept_id
  RETURNING dept_id INTO _returned;
  IF _returned IS NULL THEN
    RAISE EXCEPTION 'Department % not found', _dept_id;
  END IF;

  _returned := NULL;

  UPDATE Employees SET dept_id = _dept_id
  WHERE emp_id = _head_id
  RETURNING emp_id INTO _returned;
  IF _returned IS NULL THEN
    RAISE EXCEPTION 'Employee % not found', _head_id;
  END IF;
END;
$BODY$ LANGUAGE plpgsql;
```

▶ A DO block / function call is one atomic unit: if either `UPDATE` fails or you `RAISE EXCEPTION`, everything rolls back. Validate with `RETURNING … INTO` + a NULL check.

LECTURE

Bank transfer (guarded transaction)

Move $500 between accounts; abort if an account is missing or the balance would go negative.

```
DO $$
DECLARE _src_acc BIGINT; _dst_acc BIGINT; _src_final DECIMAL(19,2);
BEGIN
  UPDATE Account1 SET balance = balance - 500.00
  WHERE acc_no = 21301
  RETURNING acc_no, balance INTO _src_acc, _src_final;
  IF _src_acc IS NULL THEN RAISE EXCEPTION 'Source account not found'; END IF;
  IF _src_final < 0.00 THEN RAISE EXCEPTION 'Insufficient funds'; END IF;

  UPDATE Account2 SET balance = balance + 500.00
  WHERE acc_no = 40104
  RETURNING acc_no INTO _dst_acc;
  IF _dst_acc IS NULL THEN RAISE EXCEPTION 'Destination account not found'; END IF;
END $$;
```

▶ `RETURNING balance` lets you check the post-update balance before committing; any `RAISE EXCEPTION` rolls back both updates.

PLAIN SQL

Explicit transaction & rollback

```
BEGIN;
UPDATE Departments SET budget = budget - 1000 WHERE dept_id = 3;
UPDATE Departments SET budget = budget + 1000 WHERE dept_id = 5;
COMMIT;        -- or ROLLBACK; to undo everything since BEGIN
```

▶ Two statements, one unit. `PREPARE` holds a single statement only — use a function or `BEGIN/COMMIT` for two updates.

GRANT · REVOKE · COLUMN SECURITY

12 Column-Level Permissions

Hide a sensitive column (salary) by revoking blanket access and granting narrower rights — via column grants or a view.

OFFICIAL Q7 **Column privileges for salary**

Let everyone read non-salary columns; restrict salary to approved users.

```
CREATE ROLE salary_readers;

REVOKE SELECT ON Employees FROM PUBLIC;              -- drop blanket read
GRANT SELECT (emp_id, fname, lname, dept_id, manager_id)
  ON Employees TO PUBLIC;                            -- everyone: non-salary cols
GRANT SELECT (salary) ON Employees TO salary_readers; -- only this group

GRANT salary_readers TO alice;
GRANT salary_readers TO bob;
```

▶ Column-level `GRANT SELECT (cols)` exposes only the listed columns; put approved users in a role.

Q7 VARIANT **Hide salary behind a view**

```
CREATE OR REPLACE VIEW Employee_Public AS
SELECT emp_id, fname, lname, dept_id, manager_id FROM Employees;

REVOKE SELECT ON Employees FROM PUBLIC;
GRANT SELECT ON Employee_Public TO PUBLIC;           -- public sees the view
GRANT SELECT ON Employees TO salary_readers;         -- group sees the table
```

▶ The view omits `salary`; the public gets the view, trusted roles keep table access.

COMMON MISTAKES

13 Common Mistakes & Fixes

The errors the formula sheet flags most — scan these the night before.

MISTAKE	FIX
`UPDATE t (col) SET ..`	`UPDATE t SET ..` — no column list
`SET e.salary = ..`	`SET salary = ..` — no alias on the SET column
`WHERE (SELECT 1 ..)`	`WHERE EXISTS (SELECT 1 ..)`
`EXISTS` with no correlation line	add `AND d.dept_id = e.dept_id`
subquery + `GROUP BY` compared `= 0`	drop the GROUP BY, or use `NOT EXISTS`
`ALTER t ALTER col ..`	`ALTER TABLE t ALTER COLUMN c ..`
`LIKE 'research'` (expecting contains)	`ILIKE '%research%'`
`a + b` for strings	`a || b`
`col = NULL`	`col IS NULL`
`ELSEIF` / missing `END IF`	`ELSIF …` and `END IF;`
array index from 0	arrays start at **1**
aggregate in `WHERE`	move it to `HAVING`
`RETURN` used to leave a loop	`EXIT` leaves a loop; `RETURN` ends the function
repeating the target table in `UPDATE … FROM`	don't list it twice; `DELETE` uses `USING` / subquery